

Software Requirements Specification (SRS)

Project: Real-Time Weather Data Platform

Version: 1.0

Date: 2025-12-04

Prepared by: Data Platform Team

1. Introduction

1.1 Purpose

The purpose of this SRS is to define all functional, non-functional, data, and workflow requirements for the Real-Time Weather Data Platform. The system collects weather data from multiple OpenWeatherMap endpoints, stores raw data in an S3-compatible data lake, transforms it into analytic-ready structures, and loads it into a PostgreSQL-based warehouse.

This document is to be consumed by backend developers, data engineers, and DevOps engineers responsible for implementing the platform.

1.2 Scope

The system will:

- Run an automated, hourly orchestration workflow for multi-city ingestion
- Ingest **current weather**, **5-day forecasts**, and **severe weather alerts**
- Store raw JSON in a **data lake** (S3-compatible)
- Produce clean, normalized **staging** datasets in Parquet format
- Load analytics tables into PostgreSQL
- Support dashboarding via external BI tools

Out of scope (future phases):

- Real-time ingestion (<5 minute frequency)
- ML based anomaly detection
- PostGIS spatial processing
- Public-facing REST API

1.3 Definitions

Term	Definition
Data Lake	Object storage system containing raw and staged data (JSON/Parquet).
Lakehouse	Architecture pattern combining data lake + data warehouse tables.
Raw Layer	Unmodified JSON API responses stored as-is.
Staging Layer	Cleaned, normalized Parquet partitions for internal use.
Analytics Layer	Final PostgreSQL tables optimized for BI and querying.
DAG	Directed Acyclic Graph representing the workflow in Airflow/Prefect.

2. Overall System Description

The data platform follows a **modern lakehouse architecture**:

- Raw API responses stored in S3
- Python ETL performing transformations
- Parquet staging for reproducibility
- PostgreSQL warehouse for analytics
- Orchestration with Airflow or Prefect
- Optional dbt layer for further modeling

The system must support:

- **Multiple cities** (configurable)
 - **Multiple data sources** (current, forecast, alerts)
 - **Historical tracking**
 - **Automatic scheduling and retries**
-

3. Functional Requirements

3.1 Extraction Requirements

The system shall:

FR-1

Ingest data from the following OpenWeatherMap endpoints:

- /weather – current weather
- /forecast – 5-day, 3-hour intervals
- /onecall – for alerts

FR-2

Support ingestion for multiple cities defined in a configuration table.

FR-3

Write a raw JSON copy of every API response into the data lake:

data_lake/raw/{city}/{year}/{month}/{day}/{timestamp}.json

FR-4

Record API metadata including:

- latency
- status code
- payload size

FR-5

Retry failed API calls up to 3 times with exponential backoff.

3.2 Transformation Requirements

The system shall:

FR-6

Normalize all weather attributes including temperature, pressure, humidity, wind, and descriptions.

FR-7

Convert timestamps to:

- UTC
- city-specific local timezone

FR-8

Flatten nested API fields into a tabular format for:

- weather_history
- weather_forecast
- weather_alerts

FR-9

Store all transformed data in Parquet format, partitioned by:

data_lake/staging/city={city}/year={YYYY}/month={MM}/day={DD}/

FR-10

Validate raw data before loading:

- Required fields present
- Timestamps valid
- Values within safe ranges
- No duplicates

3.3 Loading Requirements

The system shall:

FR-11

Load clean data into PostgreSQL tables:

- weather_history
- weather_forecast
- weather_alerts
- cities (reference)

FR-12

Use bulk operations (COPY or batch inserts).

FR-13

Maintain historical data without overwriting.

FR-14

Ensure idempotency — no duplicate loads if the same DAG run is retried.

3.4 Orchestration Requirements

The system shall:

FR-15

Run an hourly DAG managed through **Airflow or Prefect**.

FR-16

Log run metadata:

- start/end time
- row counts
- errors and warnings

FR-17

Trigger notifications for failed runs.

3.5 Monitoring Requirements

The system shall detect:

FR-18

Missing or null fields from the API.

FR-19

Schema drift (unexpected new fields).

FR-20

Stale data (no updates for >2 hours per city).

FR-21

Row count anomalies (too few or too many entries).

4. Non-Functional Requirements

Category	Requirement
Scalability	Must support 50+ cities and >1M historical rows
Performance	End-to-end ETL must run <60 seconds for 10 cities
Reliability	99% successful DAG runs; retries on every step
Security	No credentials in code; env vars or secrets manager only
Cost Efficiency	Must operate on cloud free-tiers when possible
Extensibility	New data sources should be addable without redesign
Observability	Logs must include API, ETL, and load performance metrics

5. Data Dictionary

5.1 cities Table

Stores metadata for all tracked cities.

Column	Type	Description
id	INT PK	Unique city id
city_name	TEXT	City name
country_code	CHAR(2)	ISO-3166-1 (e.g., CA)
api_city_id	INT	OWM city ID
timezone	TEXT	IANA tz (e.g., America/Vancouver)

5.2 weather_history Table

Column	Type	Description
id	BIGSERIAL PK	Unique row
city_id	INT FK	Reference to cities
timestamp_utc	TIMESTAMP	UTC timestamp
timestamp_local	TIMESTAMP	Local timestamp
temp_c	FLOAT	Temperature
feels_like_c	FLOAT	Feels-like
pressure_hpa	INT	Pressure
humidity_pct	INT	Humidity
wind_speed_ms	FLOAT	Wind speed
weather_description	TEXT	Summary
raw_json	JSONB	Raw payload

5.3 weather_forecast Table

Same fields as history + forecast metadata.

Column	Description
forecast_timestamp	Timestamp for forecasted time
forecast_horizon	Hours ahead

5.4 weather_alerts Table

Stores severe weather alerts from OneCall API.

Column	Description
event	Alert name
sender	Issuing agency
start_utc	Start
end_utc	End
severity	Advisory/Watch/Warning
description	Alert summary

6. ETL Workflow Specification

6.1 Extract Phase

Steps:

1. Load city configuration.
 2. For each city, call /weather, /forecast, /onecall.
 3. Write each JSON response to the raw lake.
 4. Validate payload structure.
-

6.2 Transform Phase

Steps:

1. Read all raw JSON collected within the DAG run.
 2. Normalize fields (temp, wind, humidity, timestamp).
 3. Construct tabular DataFrames for each target table.
 4. Write Parquet files to the staging layer.
-

6.3 Load Phase

Steps:

1. Read Parquet files from staging.
 2. Deduplicate records.
 3. COPY into PostgreSQL.
 4. Log row counts + errors.
-

7. Acceptance Criteria

- Pipeline ingests all cities every hour
- Raw + staging + warehouse layers all populated
- No missing required fields
- No duplicate timestamps for a city

- PostgreSQL warehouse matches schema definitions in this SRS
 - DAG completes in under 60 seconds (10-city baseline)
-

8. Future Enhancements

- Near-real-time streaming using Kafka/Kinesis
- Integration of radar / satellite imagery
- Data quality governance using Great Expectations
- BI dashboards with Superset or Looker Studio
- dbt Gold Layer (semantic models)